UCL COMP0078: Supervised Learning
Problem Sheet
Tree-based learning and ensemble methods

Solutions by Shota Saito & Antonin Schrab

If you find any mistakes/typos, please contact sl-support@cs.ucl.ac.uk.

Overview of Problems

- Problem 1.1: Understanding how greedy recursive partitioning works.
- Problem 1.2: Definition and motivation for random forests.
- Problem 1.3: Understanding the differences and use cases of bagging and boosting.
- Problem 2.1: Definition and intuition for Adaboost.
- Problem 2.2: Definition/recall of how Adaboost chooses weak learners.
- Problem 2.3: Understanding the role of normalisation constants in Adaboost and how they are related to the misclassification loss.
- Problem 3: Example of sets of functions which always can/cannot classify points with 'better than random' accuracy.

Problem 1.1

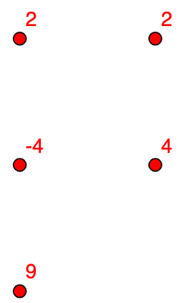
Given the dataset

$$\{\big((1,1),9\big),\ \big((1,2),-4\big),\ \big((1,3),2\big),\ \big((2,2),4\big),\ \big((2,3),2\big)\}$$

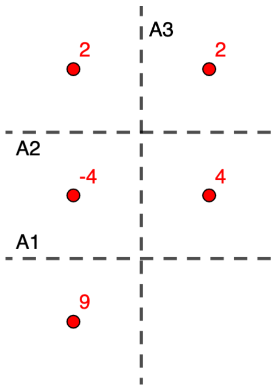
find the regression tree corresponding to the greedy recursive partitioning procedure with respect to square error.

Solution 1.1

Each element of the dataset is of the form $(\boldsymbol{x}, y)$ where the point $\boldsymbol{x} = (x_1, x_2)$ has label y.
First, let's plot the dataset:



Choice of first partition A. For the first choice, the possible partitions are:



We compute the square error for each of the three partitions, and we select the one with the lowest error.

- A1 divides the space in two regions $R_1 = \{\boldsymbol{x} : x_2 \leq 1.5\}$ with average $c_1 = 9$, and $R_2 = \{\boldsymbol{x} : x_2 > 1.5\}$ with average $c_2 = (-4 + 2 + 4 + 2)/4 = 1$. The square error is

$$\text{A1 error} = (9-9)^2 + (-4-1)^2 + (2-1)^2 + (4-1)^2 + (2-1)^2 = 36.$$

- A2 divides the space in two regions $R_1 = \{\boldsymbol{x} : x_2 \leq 2.5\}$ with average $c_1 = (9 - 4 + 4)/3 = 3$, and $R_2 = \{\boldsymbol{x} : x_2 > 1\}$ with average $c_2 = (2 + 2)/2 = 2$. The square error is

$$\text{A2 error} = (9-3)^2 + (-4-3)^2 + (4-3)^2 + (2-2)^2 + (2-2)^2 = 86.$$

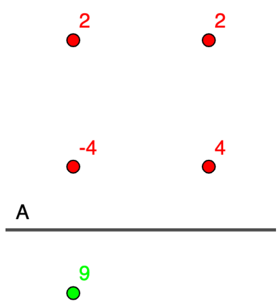
- A3 divides the space in two regions $R_1 = \{\boldsymbol{x} : x_1 \leq 1.5\}$ with average $c_1 = (9-4+2)/3 = 7/3$, and $R_2 = \{\boldsymbol{x} : x_1 > 1.5\}$ with average $c_2 = (4 + 2)/2 = 3$. The square error is

$$\text{A3 error} = \left(9 - \frac{7}{3}\right)^2 + \left(-4 - \frac{7}{3}\right)^2 + \left(2 - \frac{7}{3}\right)^2 + (4-3)^2 + (2-3)^2 \approx 86.7$$

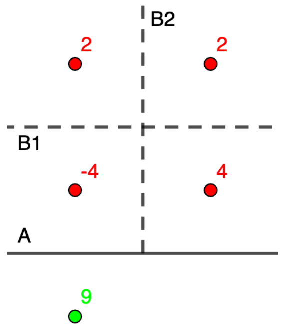
For the first partition A, we pick the partition A1 dividing the space in regions $\{\boldsymbol{x} : x_2 \leq 1.5\}$ and $\{\boldsymbol{x} : x_2 > 1.5\}$ since it has the smallest loss. This corresponds to the tree:

$$\begin{array}{c} x_2 \leq 1.5 \\ \overbrace{9 \quad ?} \end{array}$$

and the graph:



Choice of second partition B. In the region $\{\boldsymbol{x} : x_2 \leq 1.5\}$ we have error 0, so we now consider only the region $\{\boldsymbol{x} : x_2 > 1.5\}$. For the second choice, the possible partitions are:



We compute the square error for each of the two partitions, and we select the one with the lowest error.

- Within the region $\{\boldsymbol{x} : x_2 > 1.5\}$, B1 divides the space in two regions $R_1 = \{\boldsymbol{x} : x_2 \leq 2.5\}$ with average $c_1 = (-4+4)/2 = 0$, and $R_2 = \{\boldsymbol{x} : x_2 > 2.5\}$ with average $c_2 = (2+2)/2 = 2$. The square error is

$$\text{B1 error} = (-4-0)^2 + (4-0)^2 + (2-2)^2 + (2-2)^2 = 32.$$

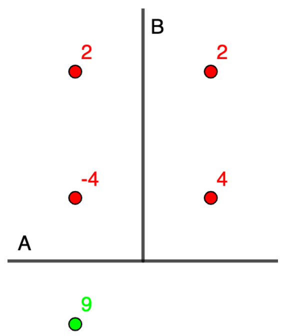
- Within the region $\{\boldsymbol{x} : x_2 > 1.5\}$, B2 divides the space in two regions $R_1 = \{\boldsymbol{x} : x_1 \leq 1.5\}$ with average $c_1 = (-4+2)/2 = -1$, and $R_2 = \{\boldsymbol{x} : x_2 > 1.5\}$ with average $c_2 = (4+2)/2 = 3$. The square error is

$$\text{B2 error} = (-4-(-1))^2 + (2-(-1))^2 + (4-3)^2 + (2-3)^2 = 20.$$

For the second partition B of the region $\{\boldsymbol{x} : x_2 > 1.5\}$, we pick the partition B2 dividing this region in subregions $\{\boldsymbol{x} : x_1 \leq 1.5\}$ and $\{\boldsymbol{x} : x_1 > 1.5\}$ since it has the smallest loss. This corresponds to the tree:

$$x_2 \leq 1.5$$
$$9 \quad x_1 \leq 1.5$$
$$? \quad ?$$

and the graph:



Choice of third partition C. Now, the region $\{\boldsymbol{x} : x_2 > 1.5 \wedge x_1 \leq 1.5\}$ has only two points, which have different labels, so there is only one possible partition C, which we select. We obtain the tree:

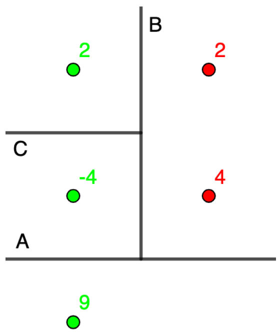
$x_2 \leq 1.5$

9 $\quad$ $x_1 \leq 1.5$

$x_2 \leq 2.5$ $\quad$?

-4 $\quad$ 2

and the graph:



Choice of fourth partition D. Now, the region $\{\boldsymbol{x} : x_2 > 1.5 \wedge x_1 > 1.5\}$ has only two points, which have different labels, so there is only one possible partition D, which we select. We obtain the tree:

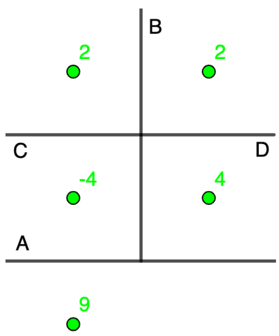
$x_2 \leq 1.5$

9 $\quad$ $x_1 \leq 1.5$

$x_2 \leq 2.5$ $\quad$ $x_2 \leq 2.5$

-4 $\quad$ 2 $\qquad$ 4 $\quad$ 2

and the graph:



Remark. The regression tree corresponding to the greedy recursive partitioning procedure with respect to the square error does not lead to the shortest tree which would be:

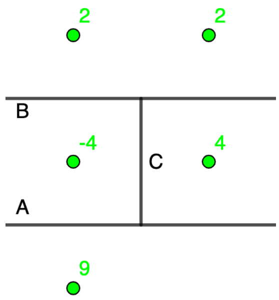
$x_2 \leq 1.5$

9 $\quad$ $x_2 \leq 2.5$

$x_1 \leq 1.5$ $\quad$ 2

-4 $\quad$ 4

corresponding to the graph:



Problem 1.2

Explain how a random forest is constructed. What is the motivation for the tree construction method?

Solution 1.2

The 'wisdom of crowds' argument for ensemble learning is strongest when each of the members of the crowds makes independent predictions. The random forest method is designed to encourage each decision tree component of the random forest to be uncorrelated.

- Draw M samples from the training data (sample with replacement)
- Consider a subset of the features from this sample ($n_{tree} < n$ where n is the total number of features - typically $n_{tree} = \sqrt{n}$ or $\log(n)$)
- Build a decision tree
- Repeat 1-2-3 N times to generate N trees
- For regression our prediction is the mean of the outputs of each tree in the random forest, for classification our prediction is the majority vote.

Both the (1) bootstrap sampling and (2) random feature selection contribute to decorrelating each tree.
Decision trees are typically high variance, so are ideal for bagging (which reduces the variance).

Problem 1.3

Both bagging and boosting produce predictions by creating an ensemble of functions. Explain how these ensembles differ (you do not need to describe the methods for arriving at the ensembles just how structurally the ensembles will differ).

Solution 1.3

Bagging considers each sample to be of equal weight but each learner only sees a subset of the data. The bias of each learner is the same as the ensemble. Our motivation is variance reduction.

Boosting places more weight on samples that previous models got wrong. Each learner sees all of the data but with variable sample weights. Learners are added an a non i.d. way to remove bias.

Problem 2.1

The Adaboost initialisation and update.

1. *Explain how Adaboost chooses its initial weighting $D_0(i)$ of the examples*
$$S = \{(\boldsymbol{x}_1, y_1), \ldots, (\boldsymbol{x}_m, y_m)\}.$$
2. *Explain how it updates the distribution D_t after choosing a weak learner h_t with coefficient α_t at stage t including the normalising constant $Z(t)$.*
3. *Give an intuitive justification for this update.*

Solution 2.1.1

It chooses an equal weighting for all examples, i.e. $D_0(i) = 1/m$ for all i.

Solution 2.1.2

The update is given by

$$D_{t+1}(i) = \frac{D_t(i)e^{-\alpha_t y_i h_t(x_i)}}{Z_t}$$

where $\alpha_t := \frac{1}{2}\log\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$, $\epsilon_t := \sum_{i=1}^m D_t(i)\mathcal{I}[h_t(\boldsymbol{x}_i) \neq y_i]$ and $Z_t := \sum_{i=1}^m D_t(i)e^{-\alpha_t y_i h_t(x_i)}$.

Solution 2.1.3

If we have a correctly classified example, $y_i = h_t(\boldsymbol{x}_i)$, so $y_i h_t(\boldsymbol{x}_i) = 1$ and then

$$D_{t+1}(i) \propto D_t(i)e^{-\alpha_t y_i h_t(\boldsymbol{x}_i)} = D_t(i)e^{-\alpha_t}.$$

Since $\alpha_t \geq 0$, the weight for that example is reduced.
Else, if we have a wrongly classified example $y_i \neq h_t(\boldsymbol{x}_i)$ so that $y_i h_t(\boldsymbol{x}_i) = -1$, then

$$D_{t+1}(i) \propto D_t(i)e^{-\alpha_t y_i h_t(\boldsymbol{x}_i)} = D_t(i)e^{\alpha_t},$$

that is, the weight for that example is increased.

Problem 2.2

Explain how Adaboost uses the distribution D_t that weights the examples $((\boldsymbol{x}_1, y_1), \ldots, (\boldsymbol{x}_m, y_m))$ to choose a weak learner from a set H at stage t.

Solution 2.2

Any hypothesis that satisfies $\mathcal{E}_t := \sum_{i=1}^m D_t(i)\mathcal{I}[h_t(\boldsymbol{x}_i) \neq y_i] < \frac{1}{2}$ will work. However, the optimal weak learner is the hypothesis that minimises $\mathcal{E}_t$.

Problem 2.3

Using the results of problem 2.1.2, obtain a bound for

$$\frac{1}{m}\sum_{i=1}^m \mathcal{I}[\text{sign}(f_T(\boldsymbol{x}_i)) \neq y_i] \leq \frac{1}{m}\sum_{i=1}^m \exp(-y_i f_T(\boldsymbol{x}_i)),$$

in terms of the normalisation constants $Z_1, \ldots, Z_T$ where $\mathcal{I}[x]$ is the indicator function with value 1 when x is true and 0 otherwise, and $f_T(\boldsymbol{x}) = \sum_{t=1}^T \alpha_t h_t(\boldsymbol{x})$.

Solution 2.3

First, we prove that

$$\frac{1}{m}\sum_{i=1}^{m}\mathcal{I}[\operatorname{sign}(f_T(\boldsymbol{x}_i))\neq y_i]\leq\frac{1}{m}\sum_{i=1}^{m}\exp(-y_if_T(\boldsymbol{x}_i)),$$

by showing that

$$\mathcal{I}[\operatorname{sign}(f_T(\boldsymbol{x}_i))\neq y_i]\leq\exp(-y_if_T(\boldsymbol{x}_i))$$

for $i=1,\ldots,m$. If $\operatorname{sign}(f_T(\boldsymbol{x}_i))=y_i$, then

$$\mathcal{I}[\operatorname{sign}(f_T(\boldsymbol{x}_i))\neq y_i]=0\leq\exp(-y_if_T(\boldsymbol{x}_i)).$$

If $\operatorname{sign}(f_T(\boldsymbol{x}_i))\neq y_i$, then $y_if_T(\boldsymbol{x}_i)\leq 0$ and so

$$\exp(-y_if_T(\boldsymbol{x}_i))\geq e^0=1=\mathcal{I}[\operatorname{sign}(f_T(\boldsymbol{x}_i))\neq y_i].$$

We are using the facts that, for $z\geq 0$, we have $0\leq e^{-z}$ and $1\leq e^{z}$.

We now express $\frac{1}{m}\sum_{i=1}^{m}\exp(-y_if_T(\boldsymbol{x}_i))$ in terms of the normalisation constants $Z_1,\ldots,Z_T$. From problem 2.1.2, we have that

$$D_{t+1}(i)=\frac{D_t(i)e^{-\alpha_ty_ih_t(x_i)}}{Z_t}.$$

This gives

$$D_{T+1}(i)=\frac{1}{m}\frac{\prod_{t=1}^{T}e^{-\alpha_ty_ih_t(x_i)}}{\prod_{t=1}^{T}Z_t}$$

We can thus write

$$\begin{aligned}\frac{1}{m}\sum_{i=1}^{m}\exp(-y_if_T(\boldsymbol{x}_i))&=\frac{1}{m}\sum_{i=1}^{m}\exp(-y_i\sum_{t=1}^{T}\alpha_th_t(\boldsymbol{x}_i))\\&=\frac{1}{m}\sum_{i=1}^{m}\prod_{t=1}^{T}\exp(-y_i\alpha_th_t(\boldsymbol{x}_i))\\&=\sum_{i=1}^{m}D_{T+1}(i)\prod_{t=1}^{T}Z_t\\&=\prod_{t=1}^{T}Z_t.\end{aligned}$$

Problem 3

A dataset $(\mathbf{x}_1,y_1),\ldots,(\mathbf{x}_m,y_m)$ is generic iff $\mathbf{x}_i=\mathbf{x}_j\implies y_i=y_j$. A set of functions $\mathcal{H}$ is called weakly-universal if for every generic data set and weighting of the data $\{D_i\}_{i=1}^m$ such that $\sum_{i=1}^m D_i=1$, there exists a function $h\in\mathcal{H}$ with weighted error $\sum_{i=1}^m D_i\,[h(\mathbf{x}_i)\neq y_i]<0.5$.

1. *A positive decision stump $h_{i,z}:\mathbb{R}^n\to\{-1,1\}$ is defined as*

$$h_{i,z}(\mathbf{x}):=\begin{cases}1 & \text{if } x_i\leq z,\\ -1 & \text{if } x_i>z.\end{cases}$$

The set of all decision stumps is defined as

$$\mathcal{H}_{ds}:=\{s\,h_{i,z}:s\in\{-1,1\},i\in\{1,\ldots,n\},z\in\mathbb{R}\}.$$

Is $\mathcal{H}_{ds}$ weakly-universal? Provide an argument supporting your answer.

2. Let

$$\mathcal{S} := \{a, b, aa, ab, ba, bb, aaa, \cdots\}$$

denote the set of all strings of non-zero finite length over $\{a, b\}$. A string x is a substring of y, denoted as $x \sqsubseteq y$ iff x is contained as a consecutive sequence of characters in y. Thus $a \sqsubseteq abba$, $ab \sqsubseteq abba$, $abba \sqsubseteq abba$ but $aba \not\sqsubseteq abba$ and $abbaa \not\sqsubseteq abba$. A positive substring-match function $g_z : \mathcal{S} \to \{-1, 1\}$ is defined as

$$g_z(x) := \begin{cases} 1 & \text{if } x \sqsubseteq z, \\ -1 & \text{if } x \not\sqsubseteq z. \end{cases}$$

The set of all substring-match functions is denoted as

$$\mathcal{H}_{\mathcal{SS}} := \{s\, g_z : s \in \{-1, 1\}, z \in \mathcal{S}\}.$$

Is $\mathcal{H}_{\mathcal{SS}}$ weakly-universal? Provide an argument supporting your answer.

Solution 3.1

We consider the set of all decision stumps

$$\mathcal{H}_{ds} := \{s\, h_{i,z} : s \in \{-1, 1\}, i \in \{1, \ldots, n\}, z \in \mathbb{R}\}$$

where $h_{i,z} : \mathbb{R}^n \to \{-1, 1\}$ is such that for all $\mathbf{x} \in \mathbb{R}^n$

$$h_{i,z}(\mathbf{x}) := \begin{cases} 1 & \text{if } x_i \le z, \\ -1 & \text{if } x_i > z. \end{cases}$$

We prove that for $\mathcal{H}^1_{ds}$ is weakly-universal and that $\mathcal{H}_{ds}$ is not weakly-universal for $n > 1$.

Consider the case of $n = 1$. Suppose we have a generic dataset $(x_1, y_1), \ldots, (x_m, y_m) \in \mathbb{R} \times \{-1, 1\}$ and some weighting of the data $\{D_i\}_{i=1}^m$ such that $\sum_{i=1}^m D_i = 1$. By relabelling the dataset if necessary, we can assume without loss of generality that $x_1 \le x_2 \le \cdots \le x_m$. Since the dataset is generic, if $\mathbf{x}_i = \mathbf{x}_j$ then $y_i = y_j$, so we can simply consider the sample only once and assign it a weight of $D_i + D_j$. Hence, without loss of generality, we can assume that all the $\{x_i\}_{i=1}^m$ are distinct and so that $x_1 < x_2 < \cdots < x_m$. Moreover, by removing data points if necessary, we can assume without loss of generality that $D_i > 0$ for $i = 1, \ldots, m$. Take $z = x_m$, so that $h_{1,x_m} \in \mathcal{H}^1_{ds}$ predicts 1 for each of $x_1, \ldots, x_m$. If the weighted error for h_{1,x_m}

$$\sum_{i=1}^m D_i \left[h_{1,x_m}(x_i) \neq y_i\right] = \sum_{i=1}^m D_i \left[1 \neq y_i\right]$$

is strictly smaller than 0.5 then we are done. If the weighted error for h_{1,x_m} is strictly greater than 0.5 then the weighted error for $-h_{1,x_m} \in \mathcal{H}^1_{ds}$ is

$$\sum_{i=1}^m D_i \left[-h_{1,x_m}(x_i) \neq y_i\right] = \sum_{i=1}^m D_i \left[-1 \neq y_i\right] = \sum_{i=1}^m D_i - \sum_{i=1}^m D_i \left[1 \neq y_i\right] = 1 - \sum_{i=1}^m D_i \left[h_{1,x_m}(x_i) \neq y_i\right] < 0.5$$

and we are done. So, assume that the weighted error for h_{1,x_m} is equal to 0.5. Recalling that $x_{m-1} < x_m$, we take $z = \frac{x_{m-1} + x_m}{2}$ so that $x_1 < x_2 < \cdots < x_{m-1} < z < x_m$. Hence, $h_{1,z} \in \mathcal{H}^1_{ds}$

predicts 1 for each of $x_1, \dots, x_{m-1}$ and -1 for x_m. We get that the weighted error for $h_{1,z}$ is

$$\begin{aligned}
\sum_{i=1}^{m} D_i \left[h_{1,z}(x_i) \neq y_i\right] &= \sum_{i=1}^{m-1} D_i \left[1 \neq y_i\right] + D_m \left[-1 \neq y_m\right] \\
&= \sum_{i=1}^{m} D_i \left[1 \neq y_i\right] - D_m \left[1 \neq y_m\right] + D_m \left[-1 \neq y_m\right] \\
&= 0.5 + D_m \Big(\left[-1 \neq y_m\right] - \left[1 \neq y_m\right] \Big) \\
&= \begin{cases} 0.5 - D_m & \text{if } y_m = -1, \\ 0.5 + D_m & \text{if } y_m = 1. \end{cases}
\end{aligned}$$

Recall that $D_m > 0$. If $y_m = -1$ then $h_{1,z} \in \mathcal{H}^1_{ds}$ has weighted error strictly smaller than 0.5. If $y_m = 1$ then $h_{1,z} \in \mathcal{H}^1_{ds}$ has weighted error strictly greater than 0.5 and so, with the same reasoning as above, $-h_{1,z} \in \mathcal{H}^1_{ds}$ has weighted error strictly smaller than 0.5. We have proved that given any dataset and any weighting of it, we can find a function in $\mathcal{H}^1_{ds}$ with weighted error strictly smaller than 0.5, hence $\mathcal{H}^1_{ds}$ is weakly-universal.

Now, consider the case of $n > 1$. We provide a counter example to prove that $\mathcal{H}_{ds}$ is not weakly-universal. Our dataset in $\mathbb{R}^n \times \{-1, 1\}$ consists of

$$\begin{aligned}
\mathbf{x}_{0,0} &:= (0, 0, 0, \dots, 0) \in \mathbb{R}^n \text{ with label } y_{0,0} := -1, \\
\mathbf{x}_{0,1} &:= (0, 1, 0, \dots, 0) \in \mathbb{R}^n \text{ with label } y_{0,1} := 1, \\
\mathbf{x}_{1,0} &:= (1, 0, 0, \dots, 0) \in \mathbb{R}^n \text{ with label } y_{1,0} := 1, \\
\mathbf{x}_{1,1} &:= (1, 1, 0, \dots, 0) \in \mathbb{R}^n \text{ with label } y_{1,1} := -1.
\end{aligned}$$

with uniform weighting $D_{0,0} = D_{0,1} = D_{1,0} = D_{1,1} := 0.25$. We need to show that for every $h \in \mathcal{H}_{ds}$, the weighted error

$$\sum_{i=0}^{1} \sum_{j=0}^{1} D_{i,j} [h(\mathbf{x}_{i,j}) = y_{i,j}] = 0.25 \sum_{i=0}^{1} \sum_{j=0}^{1} [h(\mathbf{x}_{i,j}) = y_{i,j}]$$

is exactly equal to 0.5. From this equation, we clearly see that the weighted error is exactly equal to 0.5 if and only if 2 points are correctly classified and 2 points are wrongly classified so that $0.25 \cdot 2 = 0.5$. Hence, we show by considering all cases that every $h \in \mathcal{H}_{ds}$ classifies 2 points correctly and 2 points wrongly.

For $i \in \{1, 2\}$ and $z \in (-\infty, 0) \cup [1, \infty)$, $\pm h_{i,z} \in \mathcal{H}_{ds}$ classifies all four points with the same labels, since the true labels are $\{-1, 1, 1, -1\}$ it must classify 2 correctly and 2 wrongly.

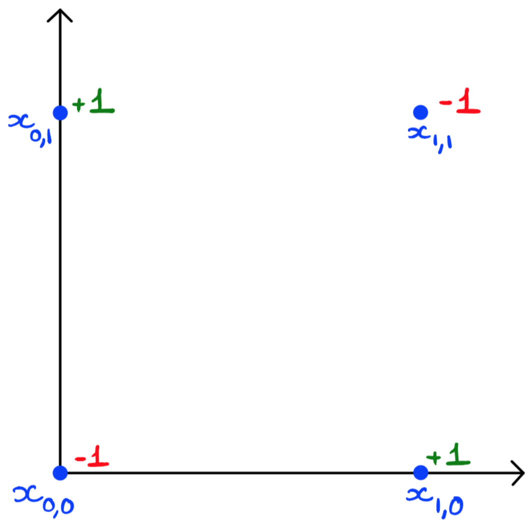
For $i = 1$ and $z \in [0, 1)$, $\pm h_{i,z} \in \mathcal{H}_{ds}$ classifies the 2 points $\mathbf{x}_{0,0}, \mathbf{x}_{0,1}$ with the same label and the 2 other points $\mathbf{x}_{1,0}, \mathbf{x}_{1,1}$ with the other label. Since $\mathbf{x}_{0,0}, \mathbf{x}_{0,1}$ have true labels $-1, 1$ and $\mathbf{x}_{1,0}, \mathbf{x}_{1,1}$ have true labels $1, -1$, we deduce that $\pm h_{i,z} \in \mathcal{H}_{ds}$ classifies 2 correctly and 2 wrongly.

Similarly, for $i = 2$ and $z \in [0, 1)$, $\pm h_{i,z} \in \mathcal{H}_{ds}$ classifies the 2 points $\mathbf{x}_{0,0}, \mathbf{x}_{1,0}$ with the same label and the 2 other points $\mathbf{x}_{0,1}, \mathbf{x}_{1,1}$ with the other label. Since $\mathbf{x}_{0,0}, \mathbf{x}_{1,0}$ have true labels $-1, 1$ and $\mathbf{x}_{0,1}, \mathbf{x}_{1,1}$ have true labels $1, -1$, we deduce that $\pm h_{i,z} \in \mathcal{H}_{ds}$ classifies 2 correctly and 2 wrongly.

If $n > 2$, for $i > 2$ and $z \in \mathbb{R}$, $\pm h_{i,z} \in \mathcal{H}_{ds}$ classifies all four points with the same labels, since the true labels are $\{-1, 1, 1, -1\}$ it must classify 2 correctly and 2 wrongly.

We have proved that every $h \in \mathcal{H}_{ds}$ classifies 2 points correctly and 2 points wrongly, and hence that every $h \in \mathcal{H}_{ds}$ has a weighted error of exactly 0.5. We have proved that $h \in \mathcal{H}_{ds}$ is not weakly-universal for $n > 1$.

Geometrically, we can plot the points projected in $\mathbb{R}^2$ and clearly see that any line perpendicular to one of the axes will either separate all four points on one side and none on the other, or separate two points with true labels $\{-1, 1\}$ on one side and two other points also with true labels $\{-1, 1\}$ on the other side.



Solution 3.2

Let $\mathcal{S}$ denote the set of all strings of non-zero finite length over $\{a, b\}$. We consider the set of all substring-match functions

$$\mathcal{H}_{\mathcal{SS}} := \{s\, g_z : s \in \{-1, 1\}, z \in \mathcal{S}\}$$

where $g_z : \mathcal{S} \to \{-1, 1\}$ is such that for all $x \in \mathcal{S}$

$$g_z(x) := \begin{cases} 1 & \text{if } x \sqsubseteq z, \\ -1 & \text{if } x \not\sqsubseteq z. \end{cases}$$

We prove that $\mathcal{H}_{\mathcal{SS}}$ is weakly-universal. Suppose we have a generic dataset $(x_1, y_1), \ldots, (x_m, y_m) \in \mathcal{S} \times \{-1, 1\}$ and some weighting of the data $\{D_i\}_{i=1}^m$ such that $\sum_{i=1}^m D_i = 1$. Since the dataset is generic, if $\mathbf{x}_i = \mathbf{x}_j$ then $y_i = y_j$, so we can simply consider the sample only once and assign it a weight of $D_i + D_j$. Hence, without loss of generality, we can assume that all the $\{x_i\}_{i=1}^m$ are distinct. By removing data points if necessary, we can also assume without loss of generality that $D_i > 0$ for $i = 1, \ldots, m$.

Take $z = x_1 \cdots x_m \in \mathcal{H}_{\mathcal{SS}}$, that is, the string obtained by concatenating all the strings in our dataset. Since $x_i \sqsubseteq z$ for $i = 1, \ldots, m$, we deduce that $h_z \in \mathcal{H}_{\mathcal{SS}}$ predicts 1 for each of $x_1, \ldots, x_m$, and so the weighted error for h_z is

$$\sum_{i=1}^m D_i \left[h_z(x_i) \neq y_i\right] = \sum_{i=1}^m D_i \left[1 \neq y_i\right].$$

If this is strictly smaller than 0.5 then we are done. If the weighted error for h_z is strictly greater than 0.5, then the weighted error for $-h_z \in \mathcal{H}_{\mathcal{SS}}$ is

$$\sum_{i=1}^m D_i \left[-h_z(x_i) \neq y_i\right] = \sum_{i=1}^m D_i \left[-1 \neq y_i\right] = \sum_{i=1}^m D_i - \sum_{i=1}^m D_i \left[1 \neq y_i\right] = 1 - \sum_{i=1}^m D_i \left[h_z(x_i) \neq y_i\right] < 0.5$$

and we are done. So, suppose that the weighted error for h_z is equal to 0.5. Let $k \in \{1, \ldots, m\}$ be such that x_k is an element of $x_1, \ldots, x_m$ with maximal length (k might not be unique). Since our dataset $x_1, \ldots, x_m$ consists of distinct strings, we must have $x_k \not\sqsubseteq x_i$ for $i \neq k$ because x_i either has smaller length than x_k or has the same length but is different. Now, consider $z = x_1 \cdots x_{k-1} x_{k+1} \cdots x_m$, that is, the string obtained by concatenating all the strings in our dataset except x_k. Then, we have $x_i \sqsubseteq z$ for $i \neq k$ and we want to have '$x_k \not\sqsubseteq z$' but this might not always be the case as maybe from concatenating some strings we could have formed the same consecutive sequence of characters as in x_k. To solve this, we can consider different permutations of $\{x_i : i = 1, \ldots, k-1, k+1, \ldots m\}$ before concatenating them. If this still does not solve our problem, then we can add extra strings from $\mathcal{S}$ in between our $x_1, \cdots, x_{k-1}, x_{k+1}, \cdots, x_m$ in the concatenation so that we indeed have $x_k \not\sqsubseteq z$. By choosing the right extra strings from $\mathcal{S}$, we can always construct z such that $x_k \not\sqsubseteq z$ and $x_i \sqsubseteq z$ for $i \neq k$. So, the weighted error of $h_z \in \mathcal{H}_{\mathcal{SS}}$ is

$$
\begin{aligned}
\sum_{i=1}^{m} D_i \left[h_z(x_i) \neq y_i\right] &= \sum_{i \neq k} D_i \left[h_z(x_i) \neq y_i\right] + D_k \left[h_z(x_k) \neq y_k\right] \\
&= \sum_{i \neq k} D_i \left[1 \neq y_i\right] + D_k \left[-1 \neq y_k\right] \\
&= \sum_{i=1}^{m} D_i \left[1 \neq y_i\right] - D_k \left[1 \neq y_k\right] + D_k \left[-1 \neq y_k\right] \\
&= 0.5 + D_k \Big(\left[-1 \neq y_k\right] - \left[1 \neq y_k\right] \Big) \\
&= \begin{cases} 0.5 - D_k & \text{if } y_k = -1, \\ 0.5 + D_k & \text{if } y_k = 1. \end{cases}
\end{aligned}
$$

Since $D_k > 0$, we find that the weighted error of $h_z \in \mathcal{H}_{\mathcal{SS}}$ is either strictly greater or strictly smaller than 0.5. If it is strictly greater than 0.5, then the weighted error of $-h_z \in \mathcal{H}_{\mathcal{SS}}$ is strictly smaller than 0.5 with the same proof as above. Hence, we have proved that for any generic dataset and any weighting of it, there exists a function $h \in \mathcal{H}_{\mathcal{SS}}$ with weighted error strictly smaller than 0.5 and so that $\mathcal{H}_{\mathcal{SS}}$ is weakly-universal.

Note that the structure of this proof is exactly the same as the one for the proof in 3.a of $\mathcal{H}^1_{ds}$ being weakly-universal. This is because the result holds more generally for some partially ordered sets, and in particular holds for $(\mathbb{R}, \leq)$ and for $(\mathcal{S}, \sqsubseteq)$.